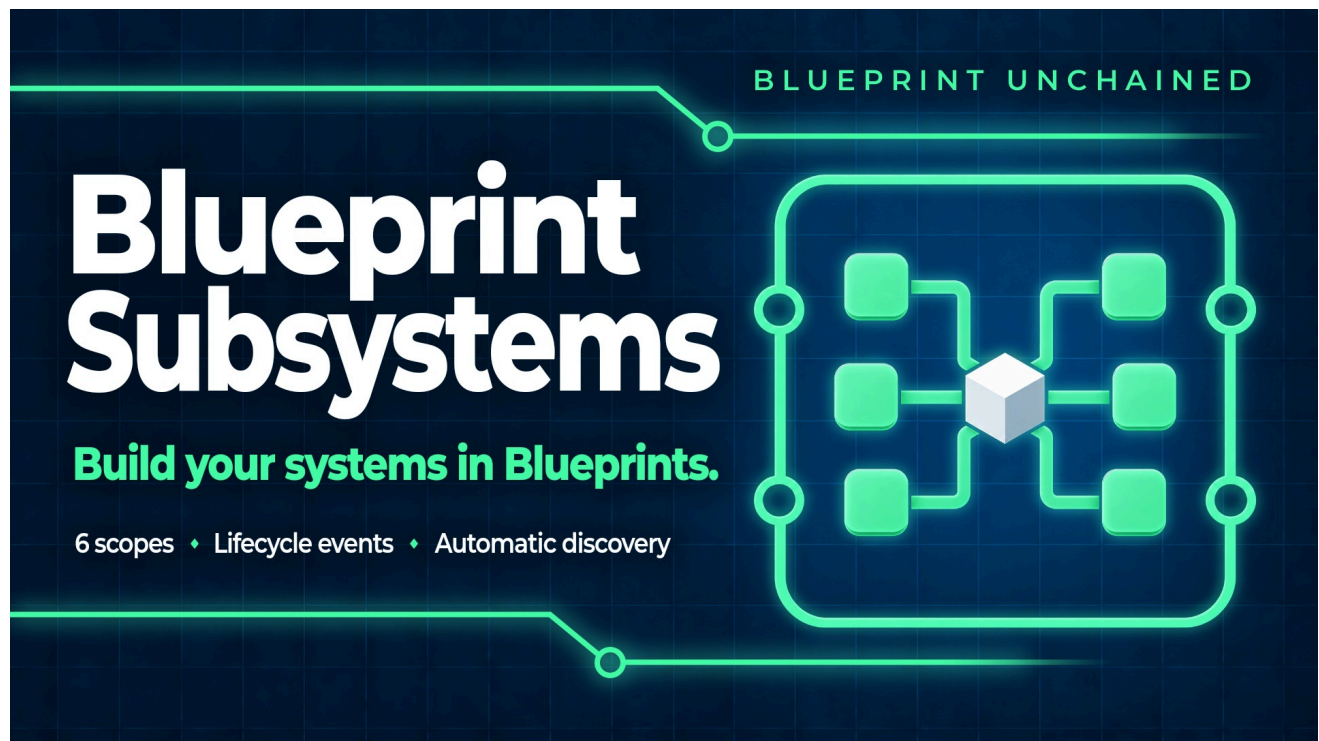


Blueprint Subsystems



Author: **Tomasz Klin**

Version: **1.0.2**

Contents

Introduction

Getting started

Choosing a scope

Engine

Editor

Game Instance

World and Tickable World

Local Player

Lifecycle

Scope-specific events

Initialize dependencies

Discovery and packaging

Runtime registration

Recompiling a subsystem

Recipes

Keep session state across level changes

Build a level service without a manager actor

Hold per-player state in split screen

Run editor validation for the whole session

Troubleshooting

My subsystem works in the editor but is missing from a build

A level opened by name is missing from a build

Get Subsystem returns no instance

Initialize runs before another subsystem is ready

Recompiling erased state

A tickable subsystem does not tick

Support

Introduction

Blueprint Subsystems lets you create shared systems entirely in Blueprint. Use one to keep quest progress between level changes, manage events in a world or store data for each local player.

A subsystem holds data and logic. Unreal Engine creates and removes it when needed. Other Blueprints access it through a **Get Subsystem** node. You do not need to place a manager actor in a map.

The plugin supports Engine, Editor, Game Instance, World, Tickable World and Local Player subsystems. It finds your subsystem Blueprints and includes them when packaging the game. Editor subsystems are available only in the editor.

Getting started

1. Enable **Blueprint Subsystems** in **Edit > Plugins**. Restart the editor if prompted.
2. In the Content Browser, choose **Add > Blueprint > Blueprint Subsystem**.
3. In **Pick Parent Class**, choose a subsystem type under **Common**, then name the new asset. Select ? beside a type to open its section of [Choosing a scope](#).
4. Open the Blueprint and add your setup logic to **Initialize**.
5. Compile and save the Blueprint.
6. In another compatible graph, search for **Get** followed by your Blueprint's class name.

To build on a subsystem Blueprint you already have, pick it under **All Classes** instead. Each subsystem Blueprint is listed there under the type it extends.

With automatic discovery enabled, the plugin finds saved subsystem Blueprints and registers them with the engine. Restart the editor after changing discovery settings that require a restart.

Choosing a scope

A subsystem's scope decides how many instances exist and how long each one lasts. Choose the scope that matches the data and logic you want to keep.

Scope	Instances	Lifetime and common uses
Engine	One per running engine process	Shared services that continue across worlds and Play In Editor sessions.
Editor	One per editor session	Editor tools and project checks. Available only in the editor.
Game Instance	One per game instance	Data for a game session, including across level changes. Useful for quest progress or matchmaking.
World	One per supported world	Data and logic for that world, such as a level event manager.
Tickable World	One per supported world	A World subsystem with a Blueprint Tick event.
Local Player	One per local player	Separate data for each local player, such as input or HUD state. Continues across level changes.

Engine

Choose **Blueprint Engine Subsystem** for a system that needs to continue when worlds or game instances change. It has no world of its own. Use a World subsystem for actors and other objects that belong to one level.

Editor

Choose **Blueprint Editor Subsystem** for editor tools and project checks. It does not exist in a packaged game, so use it only from editor code and editor Blueprints.

Game Instance

Choose **Blueprint Game Instance Subsystem** for data that lasts for one game session. It remains available between level changes and can access its owning Game Instance.

Each Play In Editor run gets a new instance, so data from one run does not carry into the next.

World and Tickable World

Choose **Blueprint World Subsystem** for a system that belongs to one world. The engine creates an instance for each supported world and removes it when that world ends.

Blueprint Tickable World Subsystem also provides a Tick event. Its settings control whether it updates in editor worlds and while the game is paused.

Use **Supported World Types** in the Blueprint defaults to choose where it runs. For a gameplay system, you will usually exclude Editor and Preview worlds.

Use the regular World type if the system only responds to events and does not need Tick.

Local Player

Choose **Blueprint Local Player Subsystem** for separate data and logic for each local player. A split-screen game has one instance per local player. A dedicated server has none.

The instance remains available when the player's controller changes or the player changes levels.

Lifecycle

Lifecycle events tell your subsystem when to start and stop its work:

- **Should Create Subsystem** runs before the instance exists. Return false to skip creating it for that owner. At this point, Self is the class default object, which holds the Blueprint's defaults. Do not read data that would belong to a running instance.
- **Initialize** runs when the instance is created. Set up data and bind to events here.
- **Deinitialize** runs before the instance is removed. Unbind events and clear references here.
- **Is Subsystem Initialized** returns true after Initialize and before Deinitialize.

Scope-specific events

Some events are available only for certain subsystem types:

- World and Tickable World receive **Post Initialize** after all subsystems in that world have initialized.
- They receive **World Begin Play** before the world's actors receive Begin Play.
- Tickable World provides **Tick** with Delta Time and **Set Tick Enabled** to control updates.
- Local Player provides **Player Controller Changed** when its controller changes.

During a level change, Player Controller Changed may first receive no controller and later receive the new controller. Check that the controller is valid before using it.

Initialize dependencies

Use **Initialize Dependency** inside Initialize if your subsystem needs another subsystem to be ready first. Both must have the same scope. The other subsystem initializes before your Initialize continues.

The order in which classes are found does not guarantee their initialization order. If you only need the other subsystem later, get it when you need it.

Discovery and packaging

Automatic discovery finds subsystem Blueprint classes and registers them with Unreal Engine. It also includes their assets when packaging the game, even when no other asset references them.

The plugin includes only subsystem Blueprints and the assets they use. A level that your game opens only by name must still be listed in the packaging settings. See [A level opened by name is missing from a build](#).

Open **Project Settings > Plugins > Blueprint Subsystems**:

- **Auto Discover Blueprint Subsystems** enables the search for subsystem Blueprints.
- **Search Paths** limits the search to folders such as `/Game/Subsystems`. Leave it empty to search all available content folders, including plugin content.
- **Additional Subsystem Classes** lists classes to register even if they are outside the search paths.
- **Register During Module Startup** registers classes early in a packaged game. Use it only when a Game Instance subsystem must be available during GameInstance initialization. It loads Blueprint assets before the engine is fully initialized.
- **Log Discovered Subsystems** records registration and unregistration in the log.
- **Reinitialize On Blueprint Compile** recreates running instances after a Blueprint compile in the editor.
- **Allow Blueprints to derive from native subsystem classes** is an experimental option for projects with existing C++ subsystem classes. Blueprints using those classes do not gain the lifecycle events provided by this plugin's base classes.

Restart the editor after changing options marked as requiring a restart.

Runtime registration

You can also control registration while the game runs. Use these nodes under **Blueprint Subsystems** for downloadable content, systems used by a specific game mode, or projects with automatic discovery disabled.

- **Register Blueprint Subsystem** registers a class and creates its instances for compatible owners that already exist. Registering the same class again returns false.
- **Unregister Blueprint Subsystem** runs Deinitialize, removes current instances and stops new owners from creating instances of that class.
- **Is Blueprint Subsystem Registered** checks whether a class is registered.
- **Get Registered Blueprint Subsystems** returns the list of registered classes.
- **Rescan Blueprint Subsystems** searches for subsystem Blueprints again and registers new classes.

If you unregister a subsystem, make sure any Blueprint that uses it can handle the instance being unavailable.

Recompiling a subsystem

Engine and Editor subsystems can remain active for the entire editor session. Enable **Reinitialize On Blueprint Compile** to apply their updated graphs after a compile.

The plugin runs Deinitialize on the affected instances and creates new ones, which run Initialize. Data held only in the old instances is lost.

This refresh is skipped during Play In Editor. Disable the option if you need to keep data in the current editor instances while editing.

Recipes

These examples show which subsystem to choose and where to put your logic.

Keep session state across level changes

Create a **Blueprint Game Instance Subsystem** and store your session data in it. Set it up in Initialize and read it from each world through its Get node.

A World subsystem is recreated when its world changes, so use Game Instance for data that must continue between levels.

Build a level service without a manager actor

1. Create a **Blueprint World Subsystem**.
2. Exclude Editor and Preview from **Supported World Types** if it is only for gameplay.
3. Use **World Begin Play** to find or spawn the level objects it needs.
4. Use **Deinitialize** to unbind events and clear references when the world ends.

Hold per-player state in split screen

Create a **Blueprint Local Player Subsystem**. Each local player gets a separate instance.

Use **Player Controller Changed** to update controller references. A level change may replace the controller.

Run editor validation for the whole session

Create a **Blueprint Editor Subsystem**. Bind to editor events in Initialize and unbind them in Deinitialize. Keep references to this class in editor tools, outside runtime gameplay graphs.

Troubleshooting

My subsystem works in the editor but is missing from a build

Check that automatic discovery can find the Blueprint or add it to **Additional Subsystem Classes**. If **Search Paths** contains folders, the asset must be in one of them unless listed separately.

Build with the source plugin. Copying only binaries generated by the editor is not enough for other build targets.

A level opened by name is missing from a build

Unreal Engine packages the maps listed in **Project Settings > Packaging > List of maps to include in a packaged build**, the default maps from **Maps & Modes**, and everything those maps use. **Open Level (by Name)** names a level without referencing it, so that level is packaged only when it is listed.

With an empty list, the engine packages all project content instead, but only when nothing asks for specific content. In Unreal Engine 5.6 to 5.8, the **NNEDenoiser** plugin is enabled by default and asks for its own content, so most projects never reach that step. Blueprint Subsystems includes your subsystems without such a request. Version 1.0.1 and earlier made one, which skipped that step in every project with a subsystem Blueprint.

To package the level, do one of the following:

- Add it to **List of maps to include in a packaged build**, in the advanced options of the **Packaging** section.
- Open it with **Open Level (by Object Reference)**. The engine follows that reference when packaging.

For other content that your game loads only by name or path, add its folder to **Additional Asset Directories to Cook**, or enable **Cook everything in the project content directory (ignore list of maps below)**.

Get Subsystem returns no instance

Check that the Get node uses the correct scope and owner. Then check **Should Create Subsystem** and **Supported World Types**.

A Local Player subsystem needs a local player; dedicated servers have none. An Editor subsystem is available only in the editor.

Initialize runs before another subsystem is ready

Call **Initialize Dependency** from Initialize when both subsystems have the same scope.

For a World subsystem that can wait until all world subsystems are ready, put the work in **Post Initialize**.

Recompiling erased state

Reinitialize On Blueprint Compile replaces the old instance with a new one. Disable the option to keep the current instance, or store data that must survive elsewhere.

A tickable subsystem does not tick

Check **Tick Enabled**, **Is Tick Enabled**, **Supported World Types** and **Tick In Editor**. To update while the game is paused, also enable **Tick When Paused**.

Support

For help or bug reports, contact **tomasz.klin@gmail.com**.

Include your Unreal Engine version, plugin version, subsystem scope and world type. This manual covers plugin version 1.0.2.